

Exercise 4 — REST API and persistence

A self-paced companion for wrapping the domain in Spring Boot and JPA

Stand-alone version of Exercise 4, for working on your own. The full design brief leaves several architecture questions open on purpose, for a classroom debate. This version settles them into concrete defaults so you can build alone in one session — and lists them at the end for you to argue with afterwards.

What you are building

Take Exercise 2's domain **unchanged** and put a real application around it: Spring Boot, a small REST API, and a database through JPA. No authentication, no security layer. Still specification-driven — the prompt stays small, and the new decisions go into the tables below instead of a long conversation.

Architecture

A plain three-layer application, by default. This isn't a ban on hexagonal/ports-and-adapters — if you already have solid, comfortable experience with it, you're welcome to use it. But the default recommendation is the plain version below: the point of this exercise is to feel why that extra complexity earns its cost, not to start there because it's the more fashionable answer. If you're unsure which to pick, ask your instructor before committing — hexagonal is a bigger investment to get right in one sitting than the exercise assumes.

COPY THIS BLOCK

HTTP request

```
-> PortfolioController    (@RestController)
-> PortfolioService       (@Service, @Transactional)
-> PortfolioRepository    (Spring Data JPA)
-> PortfolioJpaEntity <-mapper-> Portfolio (domain, unchanged)
-> H2 database (schema.sql)
```

A `@RestControllerAdvice` sits beside the controller, translating domain exceptions into HTTP responses per the error contract below.

The domain underneath, unchanged

This exercise does not regenerate the domain — it wraps it. Bring forward your own working Exercise 2 result (`Portfolio`, `Holding`, `Lot`, the value objects, the exceptions, all 36 tests still passing) unedited into this project.

If you don't have one handy, regenerate it fresh first, using Exercise 2's own prompt against `sell-stocks-spec.md` and `domain-class-diagram.puml` — then treat that result as frozen before you start on this exercise. **If you find yourself wanting to edit a domain file to make it fit Spring or JPA, stop.** That is the most interesting result this exercise can produce, and it belongs in a note, not a silent workaround.

The four endpoints

METHOD	PATH	PURPOSE	SUCCESS
POST	<code>/api/portfolios</code>	Create a portfolio	201 Created, portfolio in the body
GET	<code>/api/portfolios/{id}</code>	Fetch one, with its holdings	200 OK
POST	<code>/api/portfolios/{id}/purchases</code>	Buy shares — sets up the state a sale needs	200 OK
POST	<code>/api/portfolios/{id}/sales</code>	Sell shares. The endpoint the whole kata exists for	200 OK, proceeds/cost basis/profit in the body

No authentication, no pagination, no filtering. Adding them is a different exercise.

The error contract

Straight from Exercise 2's `error-contract.md` — this exercise is what finally turns it into running, tested behaviour instead of documentation nobody enforces.

EXCEPTION	HTTP STATUS	MESSAGE PATTERN
<code>InvalidQuantityException</code>	400	Quantity must be positive: <value>
<code>InvalidAmountException</code>	400	Price must be positive: <value>
<code>InvalidTickerException</code>	400	Invalid ticker: <value> / Ticker cannot be empty
<code>ConflictQuantityException</code>	409	Not enough shares to sell. Available: <n>, Requested: <m>

EXCEPTION	HTTP STATUS	MESSAGE PATTERN
<code>HoldingNotFoundException</code>	404	Holding not found in portfolio: <ticker>
<code>PortfolioNotFoundException</code>	404	<i>(new here — a domain-only kata had no repository to raise it)</i>

Keep the exception messages exact — they become the `detail` field of the error response, and the tests assert on them. One ordering detail carries over from the domain: quantity is validated **before** the holding is looked up, so selling zero shares of a ticker you don't hold is a 400, not a 404.

The schema

Authoritative — already written, portable, and tested against SQLite and PostgreSQL in Exercise 2. Hibernate **validates** against it; it does not generate it.

COPY THIS BLOCK

```
portfolio (portfolio_id PK, owner, balance DECIMAL(19,2), created_at)
  holding  (holding_id PK, portfolio_id FK, ticker) – unique (portfolio_id,
  ticker)
  lot      (lot_id PK, holding_id FK, initial_shares, remaining_shares,
  unit_price, purchased_at)
           – indexed on (holding_id, purchased_at) for FIFO order
```

What this schema deliberately **cannot** say, and JPA must not silently pretend it can:

- **which lot a sale consumes first** — FIFO is an ordering your code applies, not a database constraint;
- **how proceeds, cost basis, and profit are computed** — a sale returns them; nothing stores or derives them;
- **that every change goes through the portfolio** — a bare `UPDATE lot SET remaining_shares = 0` is legal SQL that nothing here refuses; and
- **that a rejected sale changes nothing** — in the domain that follows from validating before mutating; here it also needs a transaction.

Architecture decisions

Settled as defaults, so you're not debating architecture alone. Each is a legitimate one-line ADR.

DECISION	DEFAULT	WHY
Domain change	None	This exercise tests whether a carefully specified domain survives infrastructure unedited
JPA mapping	Separate <code>*JpaEntity</code> classes plus a mapper	Keeps the already-tested domain framework-free
Transaction boundary	<code>@Transactional</code> service methods, load → mutate → save	Makes “a rejected sale changes nothing” provable with a database in the loop
Aggregate loading	The whole <code>Portfolio</code> , with its holdings and lots, per request	Simplest option, acceptable at kata scale — the cost is worth discussing afterwards, not fixing now
Concurrency / locking	Out of scope	Worth discussing afterwards; not a requirement here
Schema authority	<code>schema.sql</code> is authoritative; Hibernate validates, never generates	Keeps the same “the DDL is real” stance as Exercise 2
Identifiers	App-assigned UUID strings from the domain’s <code>*Id</code> value objects	Matches the <code>CHAR(36)</code> primary keys and keeps the domain in charge of identity, not <code>@GeneratedValue</code>

H2 setup

Dependencies: `spring-boot-starter-web`, `spring-boot-starter-data-jpa`, `com.h2database:h2` (runtime scope), `spring-boot-starter-test`.

COPY THIS BLOCK

```
spring.datasource.url=jdbc:h2:mem:portfolio;DB_CLOSE_DELAY=-1
spring.jpa.hibernate.ddl-auto=validate
spring.sql.init.mode=always
spring.sql.init.schema-locations=classpath:schema.sql
```

H2 is a deliberate choice, not just a convenience: it needs nothing beyond a JAR on the class-path, so it sidesteps any dependency on Docker and Testcontainers, which this training environment may not have cleared for use yet. A real project points this same code at Postgres or MySQL later by changing the datasource URL and driver — nothing else. `schema.sql` was written to be portable for exactly this reason.

The suggested prompt

COPY THIS BLOCK

Wrap the attached, unchanged domain model in a Spring Boot REST API with JPA persistence.

Treat the attached domain source (Portfolio, Holding, Lot, the value objects, the exceptions) as immutable. If you find yourself wanting to edit one of them to make it fit Spring or JPA, stop and tell me why instead of doing it.

Build a plain three-layer application by default – controller, service, repository. (Only reach for hexagonal/ports-and-adapters if you're already comfortable with it; ask your instructor first if you're unsure.)

POST /api/portfolios	create a portfolio
GET /api/portfolios/{id}	fetch one, with its holdings
POST /api/portfolios/{id}/purchases	buy shares
POST /api/portfolios/{id}/sales	sell shares

Map exceptions to HTTP responses exactly per this table, keeping the exception messages exact:

[Paste the error contract table here.]

Use the attached schema.sql as authoritative. Configure Hibernate to validate against it, not generate it. Use H2 in-memory for now.

Follow these architecture decisions: [paste the architecture decisions table here].

Build the test suite to the scope below – do not re-derive FIFO or acceptance-criterion coverage; the domain's own 36 tests already own that. This layer only needs to prove wiring, mapping, and the error contract.

[Paste the testing scope table here.]

Done when the endpoints work end to end against H2, the error contract is enforced with matching messages, and the domain's own 36 tests still pass untouched.

The testing scope

Real tests, deliberately not exhaustive. About 19 tests total, and explicitly **not** re-verifying FIFO or AC-01...AC-24 — the domain’s existing 36 tests already own that; this layer only proves wiring, mapping, and the error contract.

LAYER	TOOL / SLICE	WHAT IT PROVES	COUNT
Mapper	Plain JUnit 5, no Spring context	Domain ↔ entity round-trip preserves values, scale, and lot order	3
Repository	<code>@DataJpaTest</code> (embedded H2)	Save/reload round-trips an aggregate; cascade delete on holding removal; unknown id returns empty	3
Service	Plain JUnit 5 + Mockito, mocked repository	One test per endpoint’s orchestration, plus the one that matters most: a rejected sale never calls <code>save()</code>	5
Controller	<code>@WebMvcTest</code> + mocked service	One happy path per endpoint, plus one representative 400 and one 404 from the exception handler	6
End to end	<code>@SpringBootTest(webEnvironment = RANDOM_PORT)</code> + <code>TestRestTemplate</code> , real H2	Full-stack wiring across all four endpoints in sequence, plus one full-stack 409 proving nothing persisted	2

Do not add: Testcontainers or a real Postgres/MySQL instance (H2 avoids the Docker dependency on purpose — see H2 setup above), RestAssured (`MockMvc` and `TestRestTemplate` already ship with `spring-boot-starter-test`), ArchUnit layering enforcement (the architecture is deliberately plain — not worth enforcing structurally at this size), optimistic-locking or retry/concurrency tests, or a performance test for aggregate-loading cost. Those are discussion items below, not test classes.

Worth discussing afterwards

- **Concurrency.** Two sales of the same holding run at once — nothing in the domain or the schema stops both from reading 15 shares and each selling 10. What would optimistic locking with a version column change, and what new acceptance criteria would it need?
- **Aggregate-loading cost.** Loading the whole `Portfolio` on every request is simple. What happens to that choice once a holding has thousands of lots?
- **JPA-on-domain vs. separate entities.** Annotating the domain directly is faster to write and couples it to a framework. What would that version have looked like here, and what would it have cost the first time the domain and the table shape needed to diverge?
- **Does “schema authoritative” survive a bigger team?** It works cleanly at kata scale. Where does that stance start to strain?

What's next

A conversation gave you working code (Exercise 1). A specification made the behaviour durable (Exercise 2). A skill and a checkpoint made the working method durable (Exercise 3). Here, that same unchanged domain had to survive contact with a database and an HTTP boundary without being edited — whether it did is the argument the whole kata was building toward.

Sell Stocks Domain Kata — self-paced companion to Exercise 4